

Lecture 03 - Geometric Transformations

This lecture explains how geometry moves between coordinate systems. Almost every later topic assumes that you can track where a point or vector currently lives.

Source chunk	course_text_parts/03_lectures/03_geometric-transformations.txt
Extracted slide pages	54
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Slide walkthrough

Each row gives a short reading comment for one extracted slide page. Use it to turn sparse slide bullets into a human-readable path through the lecture.

Page	Slide title	How to read it
1	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
2	Transforming 3D Models	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
3	3.1 Mathematical Foundations	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
4	3.1 Mathematical Foundations	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
5	Vertices as Vectors	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
6	Transformations as Matrices	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
7	Matrix Vector Multiplication	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
8	3.2 Affine Transformations	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.

Page	Slide title	How to read it
9	Translations – Moving 3D Models	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
10	Translation Properties	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
11	Scaling – Resizing 3D Models	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
12	Scaling Properties	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
13	Rotation – Changing Orientation of 3D Models 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
14	Rotation – Changing Orientation of 3D Models 2/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
15	Rotation Properties	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
16	Mirroring – Reflecting 3D Models on a Plane	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
17	Shearing – Skewing 3D Models	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
18	3.3 Composition of Transformations	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
19	Composition of Transformations	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
20	Reference Points 1/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
21	Reference Points 2/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
22	Rotation Around Arbitrary Axis	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
23	Reminder: Vector Calculus	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.

Page	Slide title	How to read it
24	Reminder: Cross Product (= vector product)	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
25	Example – Transforming Directed Line Segment 1/7	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
26	Example – Transforming Directed Line Segment 2/7	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
27	Example – Transforming Directed Line Segment 3/7	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
28	Example – Transforming Directed Line Segment 4/7	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
29	Example – Transforming Directed Line Segment 5/7	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
30	Example – Transforming Directed Line Segment 6/7	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
31	Example – Transforming Directed Line Segment 7/7	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
32	Example – Direction of Flight Transformation 1/3	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
33	Example – Direction of Flight Transformation 2/3	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
34	Example – Direction of Flight Transformation 3/3	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
35	3.4 Coordinate System Change	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
36	Coordinate System Change	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
37	Local Coordinate Systems	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
38	Example 1 – Coordinate System Change 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.

Page	Slide title	How to read it
39	Example 1 – Coordinate System Change 2/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
40	Example 2 – Hierarchical Coordinate Systems 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
41	Example 2 – Hierarchical Coordinate Systems 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
42	3.5 Transformations in OpenGL	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
43	Geometric Transformations in OpenGL	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
44	OpenGL Transformation Pipeline	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
45	Example – Translation and Scaling	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
46	Matrix Specification	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
47	Example – Portico Modeling 1/5	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
48	Example – Portico Modeling 2/5	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
49	Example – Portico Modeling 3/5	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
50	Example – Portico Modeling 4/5	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
51	Example – Portico Modeling 5/5	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
52	by applying affine transformations to their vertices	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
53	Literature and other sources used in this chapter	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

Page	Slide title	How to read it
54	(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

03.1 Mathematical Foundations

Source location: Section 3.1

Commentary

The lecture uses vectors and matrices to express geometric operations. A vector can represent a point position, a direction, an offset, or an attribute depending on context. A matrix expresses a linear map or, with homogeneous coordinates, an affine transformation.

The central exam habit is to name the coordinate space. A point in model coordinates and a light direction in view coordinates cannot be mixed safely. Many visual bugs are coordinate-space bugs.

Mental model

A numeric vector is incomplete until you know what coordinate system it belongs to.

Check yourself

Why is a normal vector not transformed exactly like a point under all transformations?

03.2 Affine Transformations

Source location: Section 3.2

Commentary

Affine transformations include translation, rotation, scaling, shearing, and combinations of these. Homogeneous coordinates allow translation to be represented in matrix form together with other transformations.

Translation moves positions but not pure directions. Rotation changes orientation while preserving lengths if it is a proper rotation. Scaling can change lengths and can also affect normals. These distinctions matter later in lighting.

Mental model

Use 4D homogeneous coordinates so that a single matrix pipeline can move 3D points through the scene.

Check yourself

What does the homogeneous w component let a transformation matrix express?

03.3 Composition of Transformations

Source location: Section 3.3

Commentary

Composition means applying several transformations in sequence. The order matters because matrix multiplication is generally not commutative. Rotating an object around its own origin and then translating it is different from translating it and then rotating around the world origin.

When you read a formula such as $\text{projection} * \text{view} * \text{model} * \text{position}$, read it from right to left for the point: first model, then view, then projection. The resulting matrix chain is compact, but the conceptual steps remain separate.

Mental model

Matrix chains are compressed stories of movement through coordinate spaces.

Check yourself

Why can swapping model and view matrices destroy the intended camera/object relation?

03.4 Coordinate System Change

Source location: Section 3.4

Commentary

A transformation can be read actively as moving an object, or passively as changing the coordinate frame used to describe it. Both interpretations are valid, but mixing them carelessly causes sign and order errors.

The view matrix is a classic example: conceptually you position a camera, but the pipeline usually transforms the world by the inverse of the camera transform so that the camera becomes the origin of view space.

Mental model

Moving the camera one way is equivalent to moving the world the opposite way for rendering.

Check yourself

Why is the view transform often related to the inverse camera transform?

03.5 Transformations in OpenGL

Source location: Section 3.5

Commentary

Modern OpenGL does not automatically manage the old matrix stacks. You usually compute model, view, and projection matrices in application code and pass them to shaders as uniforms.

The vertex shader is where the final clip-space position is normally computed. That means a wrong matrix uniform, wrong multiplication order, or wrong convention can make geometry vanish even though buffers and draw calls are correct.

Mental model

The shader is the place where abstract transformation math becomes GPU execution.

Check yourself

Which matrix would you inspect first if an object follows the camera instead of staying in the world?

End-of-lecture summary

If you remember only one thing from Lecture 03, remember this: This lecture explains how geometry moves between coordinate systems. Almost every later topic assumes that you can track where a point or vector currently lives.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.